

Railway Dashboard And Jarvis Full-Cloud Setup Test Plan

Date: 2026-07-03

Status: setup/test plan executed; provider-disabled hosted runtime verified

Scope: E1.7 hosted dashboard, Jarvis API, and agentic control surface

Executive Answer

Railway is the right backend host for runtime features that Vercel static/serverless review packets cannot honestly provide: long-running workers, queues, provider calls, SSE/websocket events, durable uploads, persistent run state, auth, and controlled writeback.

Current proven state is Vercel-connected and provider-disabled, with a Railway-hosted Jarvis API now verified for the review-packet contract. The deployment proves the smallest cloud runtime for dashboard-to-backend operation without local Codex/Jarvis. It does not activate provider models, autonomous writeback, raw voice storage, or customer-facing auth/persistence.

Live Railway Check On 2026-07-03

FACT:

- Railway MCP was reachable, but not link-aware for this local project in the active session.
- Railway CLI was linked to the project/service and used for service status, variables, deployment, domain generation, and logs.
- Only non-secret provider-disabled runtime variables were set.
- The service root deployed was `services/jarvis-api`.
- The hosted service reached running success state and passed HTTPS endpoint checks.

GAP:

- Railway URL, deployment IDs, project IDs, and account-identifying details are intentionally excluded from this public report.
- Authentication, persistent storage, provider activation, and durable writeback remain outside this E1.7 provider-disabled baseline.

Current Cloud Readiness

Layer	Current proof	Status
Static dashboard	Vercel-hosted dashboard plus browser-local API-base control for a hosted backend	Working for review packets
Local Jarvis API	FastAPI contract under <code>services/jarvis-api/</code> plus in-process smoke test	Passed
Railway	<code>jarvis-api</code> service deployed from <code>services/jarvis-api</code> with provider-disabled variables	Passed
Provider calls	<code>MODEL_PROVIDER=none</code> default; provider routes disabled	Gated

Layer	Current proof	Status
Writeback	Git/Notion/WikiLLM/Telegram dashboard writeback disabled	Gated
Voice	Browser-local controls and text path only	Gated for raw audio/storage

Target E1.7 Definition

E1.7 means: hosted dashboard, Jarvis API, and agentic system can run the provider-disabled review-packet contract without relying on the local machine.

It does not mean provider-backed automation, autonomous Notion/GitHub/WikiLLM writeback, raw voice storage, authenticated client portals, or paid customer runtime.

Deployment Architecture

```
Vercel static dashboard
-> configured backend URL
-> Railway jarvis-api service
-> provider-disabled review packet endpoints
-> logs/status packets
-> Codex/owner review before any writeback
```

Railway Setup Plan

Step	Action	Check	Gate
R1	Confirm Railway project/service from prior setup or link/create project	railway status or Railway MCP project/service list	Owner approval for cloud work
R2	Deploy only services/jarvis-api	Railway deploy logs show build/start	Approval for code upload
R3	Set env variables from placeholders	MODEL_PROVIDER=none, allowed origin, budget defaults	No secrets in repo
R4	Verify health	GET /health returns ok, provider calls 0, writeback disabled	Required before dashboard routing
R5	Verify CORS/auth boundary	Allowed origin works; disallowed origin blocked where applicable	Required before public URL use
R6	Exercise endpoints	chat, roles, PRD/ICP, agent-orchestra, voice text route return review packets	Provider disabled
R7	Point dashboard to Railway test backend	dashboard state shows connected and no provider calls	No production promotion
R8	Record run	run note, PDF, dashboard data, wiki log, validation	Public safety scan

Executed Endpoint Test Matrix

Endpoint	Result	Evidence meaning
/health	Passed	service ok, model provider none, provider calls 0, external writeback 0
CORS preflight	Passed	Vercel dashboard origin can POST to the hosted API
/api/chat	Passed	review packet, no provider output
/api/config/roles	Passed	role config payload available for dashboard configuration

Endpoint	Result	Evidence meaning
/api/lanes/prd-icp	Passed	Architecture 1 PRD/ICP cloud route works
/api/lanes/agent-orchestra	Passed	Architecture 2 agent-orchestra cloud route works
/api/voice/chat	Passed	text review packet only; no raw audio storage
Railway deploy logs	Passed	Uvicorn started and listened on the assigned port
Railway HTTP error logs	Passed	no recent hosted HTTP error entries returned for the validation probes

Dashboard Routing Update

The dashboard now includes a browser-local Jarvis API base setting. The operator can paste the hosted backend origin, save it to local browser storage, check /health, and send provider-disabled packets to the correct lane:

Dashboard mode	Backend endpoint	Purpose
Architecture 1 / service	/api/lanes/prd-icp	PRD/ICP review packet
Architecture 2 / control	/api/lanes/agent-orchestra	agent role/configuration review packet
Config	/health	hosted runtime readiness

The browser setting is not a secret and is not committed as environment state.

Provider Activation Gate

Provider activation is a later lane. Required before any model call:

- server-side secrets only;
- current model list and pricing checked;
- ledger writes for model, input class, output class, token estimate, spend estimate;
- daily cap and run hard stop active;
- sanitized input class approved;
- AF Review and owner approval recorded;
- no raw private transcript, credentials, private URLs, local paths, or account IDs sent.

What Full Cloud Setup Provides

For ArchFlow:

- a durable provider-disabled operator API when local Codex/Jarvis is off;
- a cloud review-packet API for PRD/ICP and Agent Orchestra lanes;
- a future bridge for provider-backed tasks after budget/safety gates;
- a cleaner demo of the operating system behind the PRD Pack.

For clients:

- a visible method for turning source material into reviewed artifacts;
- stronger confidence that outputs are gated, traceable, and not raw AI chat;
- a future path toward secure client portals and artifact delivery.

Critique

The current setup is useful but fragile if described as a finished product. It is an operator control system plus service-delivery proof with a small hosted runtime, not yet a customer-ready cloud app. Railway has now proved backend reachability and policy enforcement for provider-disabled packets. Productization should still wait until E2/E6/E7 prove buyer demand and until auth, persistence, and provider budget ledgers are implemented.

Acceptance Criteria

Criterion	Result
Railway deployment uses only services/jarvis-api	Passed
/health passes from the Railway domain	Passed
Provider calls remain zero	Passed
Writeback remains disabled	Passed
Dashboard can store/check hosted API base	Passed locally and browser-verified on the production dashboard Config screen; latest E1.7 dashboard data remains preview-current before production freshness proof
Public safety scan, runtime guard, dashboard JSON parse, and diff check pass	Passed in E1.7 closeout; rerun required after the later cloud/KB retrospective files
Run handout and Notion/GitHub/Telegram packets are created	Passed for E1.7 closeout; later cloud/KB retrospective adds a separate packet